

AutoJudge – Problem Difficulty Predictor

Ainesh Kumar Das (24114008)

1 Introduction

This Project aims to develop an automated system in order to classify the difficulty level of a programming task as easy, medium or hard and also a problem difficulty score to make comparisons within a class.

2 Problem Statement

The objective of this project is to:

- Perform accurate classification of problem difficulty levels.
- Predict a continuous difficulty score using regression techniques.
- Deploy the trained model through a user-friendly web interface.

3 Dataset Description

The dataset used in this project is derived from the research paper:

“TaskComplexity: A Dataset for Task Complexity Classification with In-Context Learning, FLAN-T5 and GPT-4o Benchmarks”, arXiv:2409.20189

The dataset provides a comprehensive collection of programming problems, each labeled by classification, enabling the development and evaluation of models based on task difficulty.

3.1 Dataset Characteristics

- Number of samples: 4112
- Number of features: 6 (Task Title, Problem Description, Input/Output Specification, Examples, Problem Class, Complexity Score)

- Target variables:
 - Classification label (Easy / Medium / Hard)
 - Regression score (continuous difficulty value)

4 Data Preprocessing

Problem statements of programming tasks are often written in LaTeX and, when extracted they contain residual LaTeX commands and symbols. The aim of data preprocessing was to produce clean and standardized data.

To achieve this, the following steps were performed during data preprocessing:

- Missing values in all textual fields (`title`, `description`, `input_description`, and `output_description`) were replaced with empty strings and combined into a single string, referred to as `combined_text`, for further analysis.
- All text was converted to lowercase and HTML tags were removed.
- Common LaTeX mathematical symbols used in constraints (such as `\leq`, `\geq`, and `\times`) were normalized into their symbolic equivalents (e.g., `<=`, `>=`, `*`).
- All inline math delimiters have been removed, but numerical bounds and operators have been preserved such that important complexity constraints, like $1 \leq n \leq 10^5$ remain interpretable.
- Any remaining LaTeX commands were stripped, and extra whitespace was normalised.

The following preprocessing steps are implemented in the `preprocess.py` module and prepare the cleaned, standardized textual representation ready for both TF-IDF-based text feature extraction and manual feature engineering.

5 Feature Engineering

Feature engineering was performed to capture both textual and structural characteristics of programming problem statements.

5.1 Techniques Used

- **TF-IDF text features:** The combined problem text was vectorized using a TF-IDF representation with unigrams and bigrams, capturing important terms and phrases relevant to problem difficulty.

- **Statistical text features:** Simple features such as total text length and the number of mathematical symbols were extracted to estimate the problem difficulty with the assumption that difficult tasks generally have higher text length and have more mathematical symbols in the statement.
- **Keyword-based features:** In programming, the occurrence of certain words usually implies a higher difficulty (e.g., graphs, trees, bitmasking). I came up with a list of keywords that generally come up in the statement of medium-hard problems, the frequency of these keywords were also used in determining the toughness of a problem.

The final feature representation consists of a concatenation of TF-IDF text features and manually engineered numerical features. All feature extraction steps were implemented in the `features.py` module.

6 Models Used

6.1 Classification Model

A **Random Forest Classifier** was employed to predict the difficulty class of programming problems. Class imbalance was handled using balanced class weights, and multiple decision trees were used to improve robustness and generalization.

6.2 Regression Model

A **Random Forest Regressor** was used to estimate a continuous difficulty score for each problem.

Random Forest models were selected due to their robustness, resistance to overfitting, and effectiveness in handling high-dimensional feature spaces generated from textual data.

7 Experimental Setup

7.1 Training Configuration

- Train-test split: 80% / 20%
- Stratified sampling based on difficulty classes
- Evaluation performed on unseen test data

7.2 Implementation Details

- Language: Python
- Libraries: Scikit-learn, NumPy, Pandas, Joblib, Streamlit

Model training logic is implemented in `train.py`.

8 Results and Analysis

8.1 Classification Results

Table 1: Classification Performance

Metric	Value
Accuracy	50.91%
Precision	0.5015
Recall	0.5091
F1-score	0.4858

8.2 Confusion Matrix

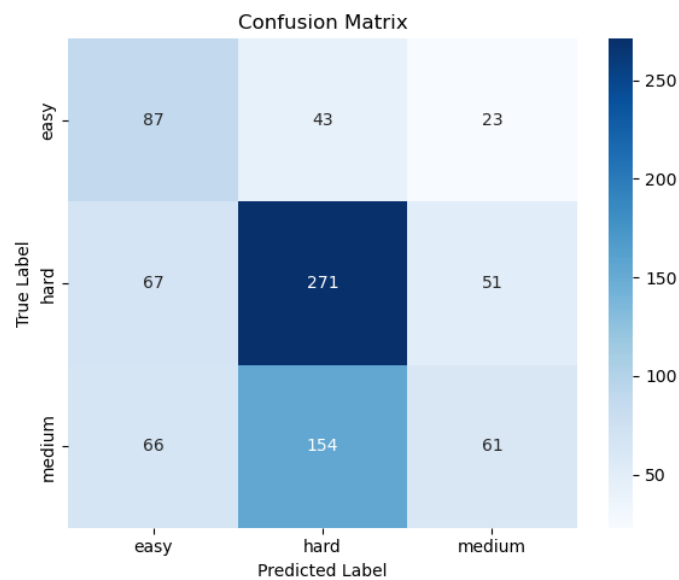


Figure 1: Confusion Matrix for Classification

8.3 Regression Results

Table 2: Regression Performance

Metric	Value
MAE	1.706
RMSE	2.034

9 Web Interface

A web-based interface was developed using **Streamlit** to allow users to input problem statements and obtain predictions in real time.

9.1 Features

- Text input areas for problem description, input description, and output description
- Predicts both the difficulty class (classification) and difficulty score (regression) using the trained models
- Lightweight and responsive interface, with immediate feedback upon submission
- Models are loaded efficiently using caching to ensure fast predictions

9.2 Sample Predictions

AutoJudge – Problem Difficulty Predictor

Problem Description

You are given all numbers between $1, 2, \dots, n$ except one. Your task is to find the missing number.

Input Description

The first input line contains an integer n .
The second line contains $n-1$ numbers. Each number is distinct and between 1 and n (inclusive).
 $2 \leq n \leq 2 \cdot 10^5$

Output Description

Print the missing number.

Predict

Predicted Difficulty Class: easy

Predicted Difficulty Score: 4.02

AutoJudge – Problem Difficulty Predictor

Problem Description

The goal is to move all the disks to the right stack using the middle stack. On each move you can move the uppermost disk from a stack to another stack. In addition, it is not allowed to place a larger disk on a smaller disk.
Your task is to find a solution that minimizes the number of moves.

Input Description

The only input line has an integer n : the number of disks.
 $1 \leq n \leq 16$

Output Description

First print an integer k : the minimum number of moves.
After this, print k lines that describe the moves. Each line has two integers a and b : you move a disk from stack a to stack b .

Predict

Predicted Difficulty Class: medium

Predicted Difficulty Score: 4.01

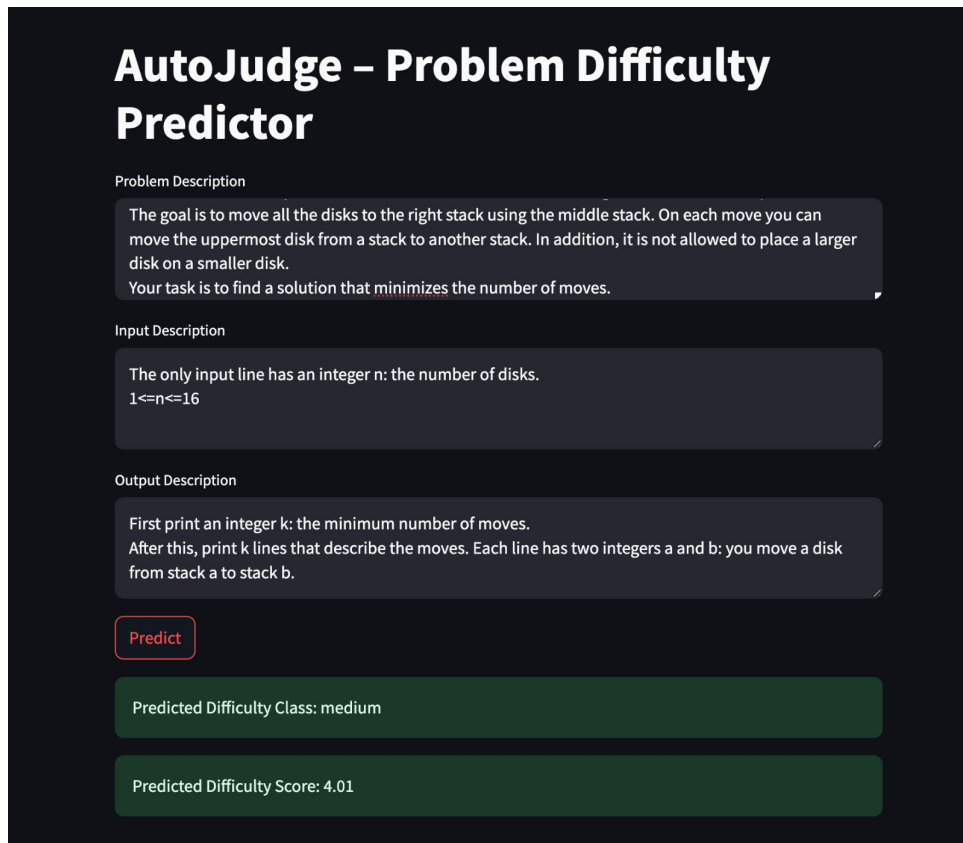


Figure 2: Web Interface Showing Sample Prediction

The web logic is implemented in `app.py`.

10 Conclusion

This work includes the use of textual analysis and feature engineering to make a prediction related to the difficulty level of programming tasks. The system uses a combination of TF-IDF text features and manually calculated numerical features in performing classification and regression.

The Random Forest classifier achieved an accuracy of 50.91% in predicting difficulty classes, while the regressor obtained an MAE of 1.706 and an RMSE of 2.034 for continuous difficulty scores.

The web interface built with Streamlit provides a user-friendly platform for real-time predictions, allowing users to interactively input problem descriptions and receive both difficulty class and score.